

Cuaderno 2 - Repaso de Pandas, Matplotlib, Seaborn y Plotly

Bienvenidos al segundo cuaderno. En este recorrido vamos a hacer un repaso integral de las herramientas fundamentales que usamos para manipular y visualizar datos en Python. Vamos a volver a trabajar algunos datasets de **Kaggle** para poner todo en práctica.

En la segunda parte, analizaremos rápidamente un dataset diferente, y crearemos algunas visualizaciones más, para terminar con esta etapa de revisión de conceptos básicos antes de comenzar con los temas centrales de **Machine Learning**.

Importación de librerías necesarias

Para trabajar cómodamente, necesitamos importar cuatro pilares del análisis de datos. Acá te explico para qué sirve cada una:

- **Pandas:** Es nuestra herramienta principal para trabajar con tablas (DataFrames).
- **Matplotlib** y **Seaborn:** Son nuestras aliadas para crear gráficos estáticos, elegantes y fáciles de personalizar.
- **Plotly:** La vamos a usar cuando necesitemos gráficos interactivos (donde podés hacer zoom o pasar el mouse para ver valores exactos).

Además, vamos a configurar el entorno para que todos los gráficos tengan un estilo limpio y profesional.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import kagglehub # Acceso a Kaggle
import os        # Para las rutas de los archivos.

# Configuramos el estilo de Seaborn con fondo blanco
# y cuadrícula para leer mejor los datos
sns.set(style='whitegrid')

# Ajustamos el tamaño estándar de los gráficos
# (8 pulgadas de ancho por 5 de alto)
# Esto sirve para que no salgan ni muy chiquitos ni deformados.
plt.rcParams['figure.figsize'] = (8, 5)
```

Carga del dataset

Volvemos a cargar nuestro querido 'Superstore Sales Dataset'.

- **Fuente del Dataset:** [Sales Forecasting \(Kaggle\)](#)

Como ya tenemos el archivo `train.csv` en nuestra carpeta de trabajo, simplemente lo leemos con `pandas` y guardamos todo en un **DataFrame** llamado `df`. Usamos `df.head()` para verificar que los datos cargaron bien y ver cómo se ven las primeras filas.

```
# Intentamos abrir el archivo si ya está disponible en el entorno
try:
    df = pd.read_csv("train.csv")
    print("Dataset cargado desde el archivo local 'train.csv'")
except Exception:
    print("No se encontró 'train.csv' en el entorno local.")
    print("Descargando dataset desde Kaggle...")

    ruta_dataset = kagglehub.dataset_download("rohitsahoo/sales-forecasti
    ruta_csv = os.path.join(ruta_dataset, "train.csv")

    df = pd.read_csv(ruta_csv)
    print("Dataset descargado y cargado correctamente desde Kaggle")
```

No se encontró 'train.csv' en el entorno local.
Descargando dataset desde Kaggle...
Using Colab cache for faster access to the 'sales-forecasting' dataset.
Dataset descargado y cargado correctamente desde Kaggle

Pandas - Exploración y limpieza básica

Ahora que ya cargamos los datos, necesitamos "interrogarlos" para entender qué tenemos entre manos. Esta etapa es clave para saber si los datos están limpios o si vamos a tener que trabajar un poco antes de pasar a la parte de Machine Learning.

Vamos a usar nuevamente los comandos esenciales de `pandas` :

```
# 1. Información general: tipos de datos y cantidad de valores no nulos
print("Tipos de datos y cantidad de valores no nulos:\n")
df.info()
print()
print("-"*60)
```

```
# 2. Conteo de valores nulos: para ver cuántos datos nos están faltando
```

```
print("Conteo de valores nulos:\n")
df.isnull().sum()
print()
print("-"*60)
```

*# 3. Resumen estadístico: vemos medias, máximos, mínimos y más
(solo sobre columnas numéricas)*

```
print("Resumen estadístico:\n")
df.describe()
print()
print("-"*60)
```

4. Filtrado: nos quedamos solo con las ventas mayores a 1000

```
print("Ventas mayores a 1000:\n")
df[df['Sales'] > 1000].head()
print()
print("-"*60)
```

5. Verificamos cuál es el valor más alto en la columna Sales

```
print("La venta más alta registrada es de:", df['Sales'].max())
```

Tipos de datos y cantidad de valores no nulos:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9800 entries, 0 to 9799
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                 9800 non-null  int64
1   Order ID               9800 non-null  object
2   Order Date             9800 non-null  object
3   Ship Date              9800 non-null  object
4   Ship Mode               9800 non-null  object
5   Customer ID            9800 non-null  object
6   Customer Name          9800 non-null  object
7   Segment                9800 non-null  object
8   Country                9800 non-null  object
9   City                   9800 non-null  object
10  State                  9800 non-null  object
11  Postal Code            9789 non-null  float64
12  Region                 9800 non-null  object
13  Product ID             9800 non-null  object
14  Category                9800 non-null  object
15  Sub-Category           9800 non-null  object
16  Product Name           9800 non-null  object
17  Sales                  9800 non-null  float64
dtypes: float64(2), int64(1), object(15)
memory usage: 1.3+ MB
```

Conteo de valores nulos:

Resumen estadístico:

Ventas mayores a 1000:

La venta más alta registrada es de: 22638.48

¿Qué nos muestran estos comandos?

- `df.info()` : Es como el "DNI" del dataset. Te dice cuántas filas tenés, qué columnas hay y, lo más importante, si Python las reconoce como números, texto (objetos) o fechas.
- `df.isnull().sum()` : Fundamental. Si una columna tiene muchos valores nulos, el modelo de Machine Learning puede fallar o dar resultados erróneos. Acá identificamos esos "huecos".

- `df.describe()` : Te da un pantallazo estadístico. Si ves que el valor mínimo de `Sales` es negativo, por ejemplo, ya sabés que tenés datos que no tienen sentido y vas a tener que revisarlos.
- `df[df['Sales'] > 1000]` : Esta es la forma en la que le pedimos a pandas que filtre. Estamos haciendo una pregunta: "*¿Cuáles son las filas donde la columna Sales es mayor a 1000?*". Es una herramienta muy poderosa para encontrar casos particulares o valores extremos (`outliers`).

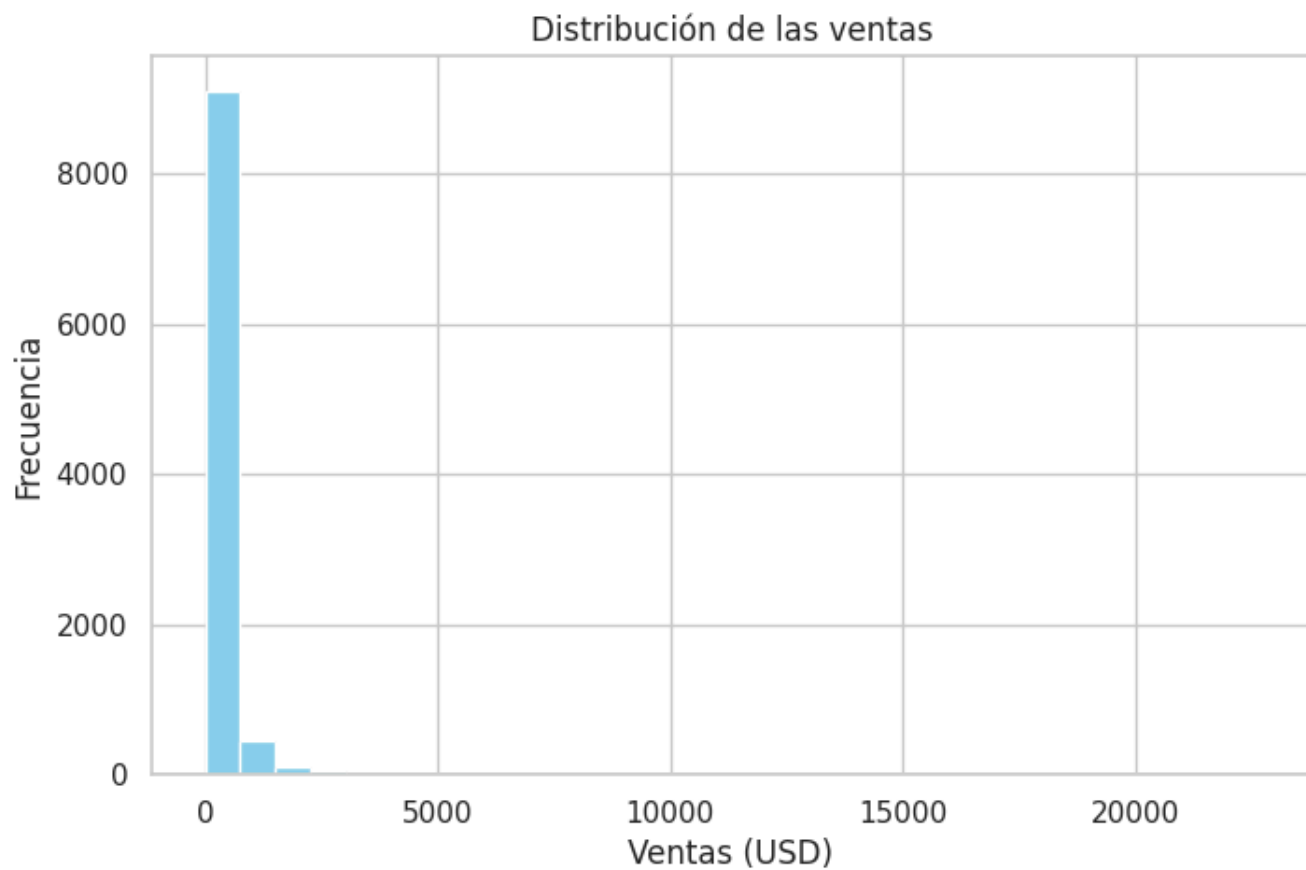
Si querés ver estos temas un poco más en profundidad, podés darle un vistazo a los cuadernos anteriores, de [Análisis de Datos con Python](#)

Matplotlib - Gráficos básicos

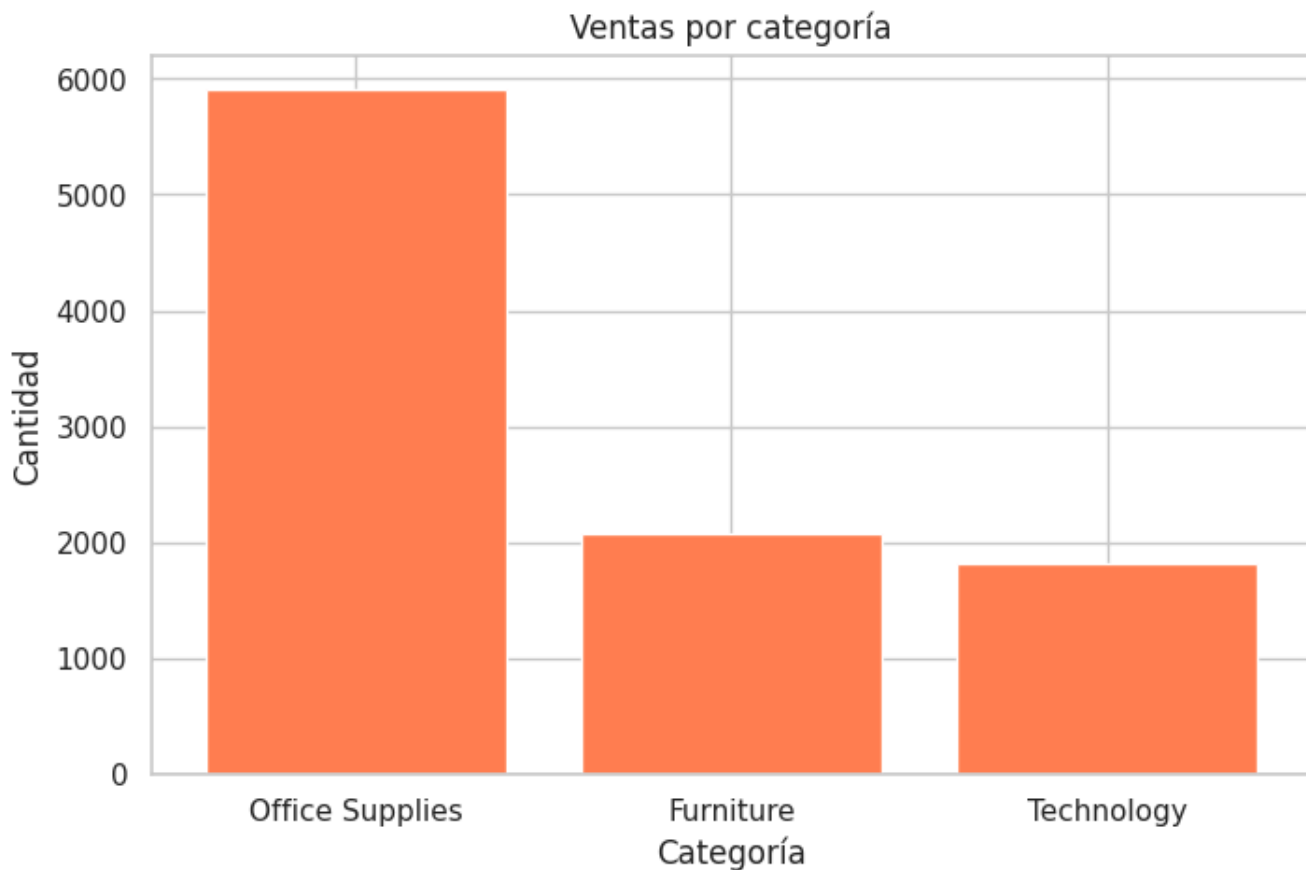
A veces, mirar números en una tabla no nos dice mucho. Para entender realmente qué está pasando con los datos, lo mejor es visualizarlos. Vamos a usar **Matplotlib**, la librería más clásica y versátil para crear gráficos en Python.

Acá vamos a ver dos tipos de gráficos fundamentales:

```
# 1. Histograma: Para ver cómo se distribuyen nuestras ventas
plt.hist(df['Sales'], bins=30, color='skyblue')
plt.title('Distribución de las ventas')
plt.xlabel('Ventas (USD)')
plt.ylabel('Frecuencia')
# usar plt.xlim(min, max) para hacer "zoom"
plt.show()
```



```
# 2. Gráfico de barras: Para comparar cantidades entre categorías
cat_counts = df['Category'].value_counts()
plt.bar(cat_counts.index, cat_counts.values, color='coral')
plt.title('Ventas por categoría')
plt.xlabel('Categoría')
plt.ylabel('Cantidad')
plt.show()
```



¿Para qué nos sirve cada uno?

- El **Histograma** (`plt.hist`): Es la mejor forma de ver la "forma" de tus datos. Al usar `bins=30` , estamos dividiendo el rango de ventas en 30 intervalos. Esto nos permite ver si la mayoría de las ventas son pequeñas, si hay muchas ventas iguales o si existen valores extremos muy altos que se alejan del resto.
- El **Gráfico de Barras** (`plt.bar`): Es ideal para comparar categorías. Primero usamos `value_counts()` para contar cuántas filas hay de cada categoría, y luego lo graficamos. Es la forma más rápida de responder preguntas como: *"¿Qué tipo de producto se vende más?"*.

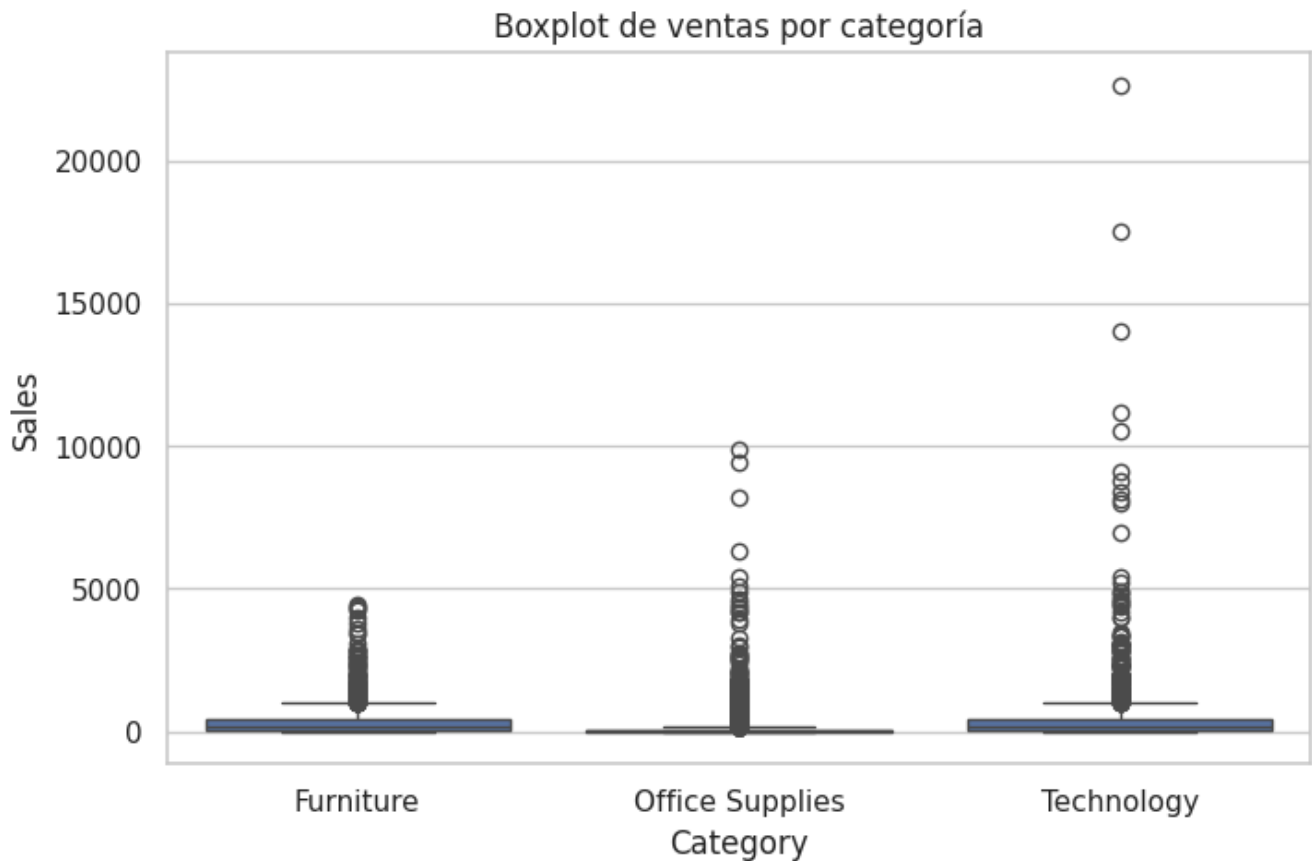
Para ampliar estos conceptos, podés volver a consultar el cuaderno ["Visualización con Matplotlib"](#)

Seaborn - Visualizaciones estadísticas

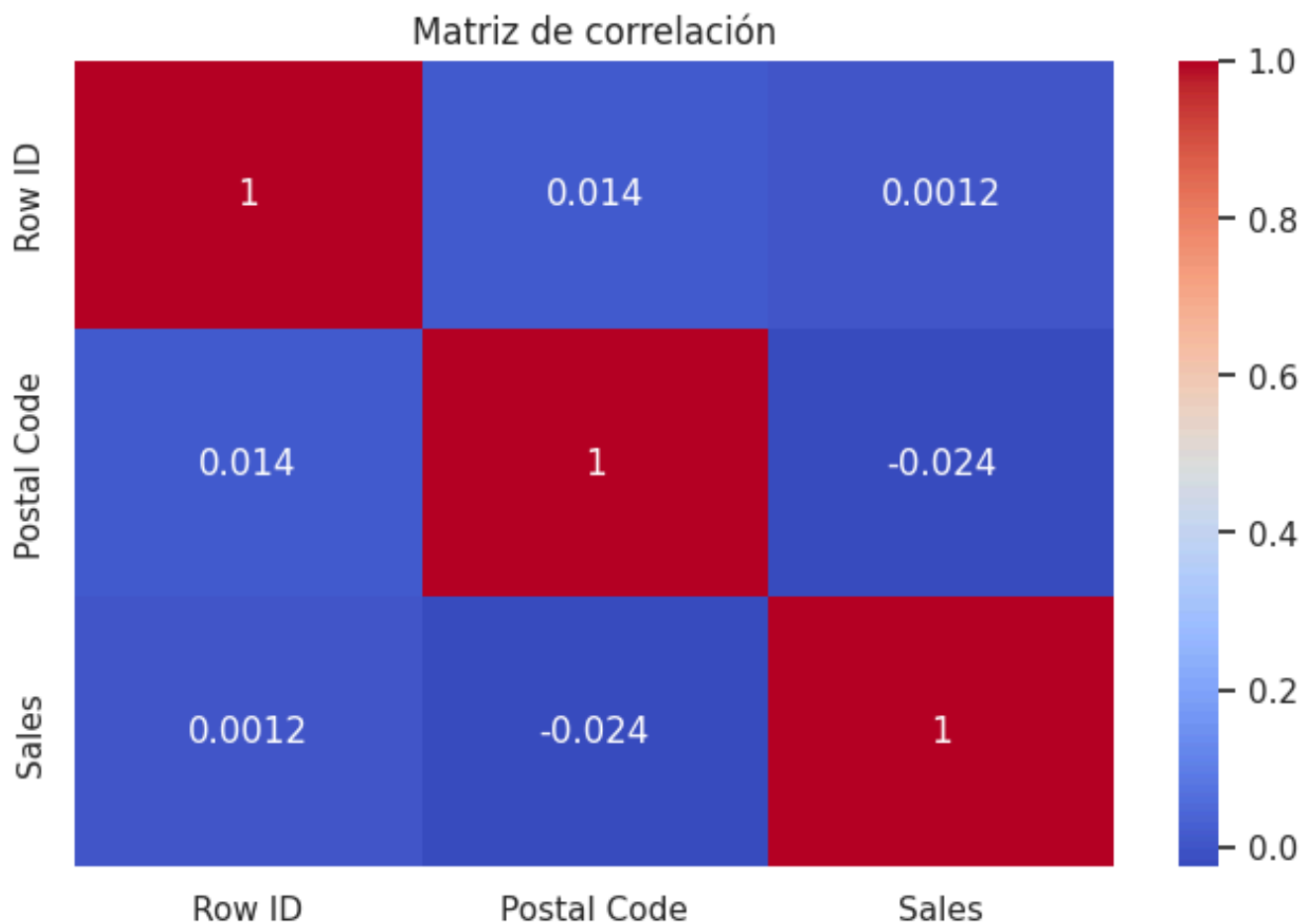
Ahora vamos a dar un salto de calidad con **Seaborn**. Esta librería está construida sobre **Matplotlib**, pero está diseñada específicamente para estadísticas y gráficos más complejos (y un poco mas modernos esteticamente) con menos líneas de código.

Vamos a realizar dos análisis muy importantes:

```
# 1. Boxplot (Diagrama de caja):  
# Para ver la dispersión de las ventas por categoría  
sns.boxplot(x='Category', y='Sales', data=df)  
plt.title('Boxplot de ventas por categoría')  
plt.show()
```



```
# 2. Heatmap (Mapa de calor):  
# Para entender la relación entre las variables  
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')  
plt.title('Matriz de correlación')  
plt.show()
```

¿Qué estamos mirando en estos gráficos?

- El **Boxplot**: Es uno de los gráficos más importantes en Machine Learning. Nos muestra la **mediana** (la línea dentro de la caja), los cuartiles (donde se concentra el grueso de los datos) y, sobre todo, esos puntitos que sobran arriba: los valores atípicos o outliers. Si antes te preguntabas por qué el gráfico de ventas se "estiraba" tanto, acá vas a ver a los culpables.
- El **Heatmap de correlación**: Este gráfico nos dice qué tanto se "parecen" o se influyen dos variables entre sí.

- Un valor cercano a 1 (color cálido/rojo) significa que cuando una variable sube, la otra también.
- Un valor cercano a -1 (color frío/azul) significa que cuando una sube, la otra baja.
- Un valor cercano a 0 significa que no hay una relación clara.

Nota: Usamos `numeric_only=True` porque no tiene sentido calcular correlaciones entre palabras (como nombres de ciudades), solo lo hacemos entre números.

Te recomiendo ver el [Cuaderno con Ejemplos Prácticos de Seaborn](#) y el [Cuaderno de Gráficos con Seaborn](#) del curso de **Ciencias de Datos con Python** para ampliar este repaso.

Seaborn: Countplot (Conteo de registros)

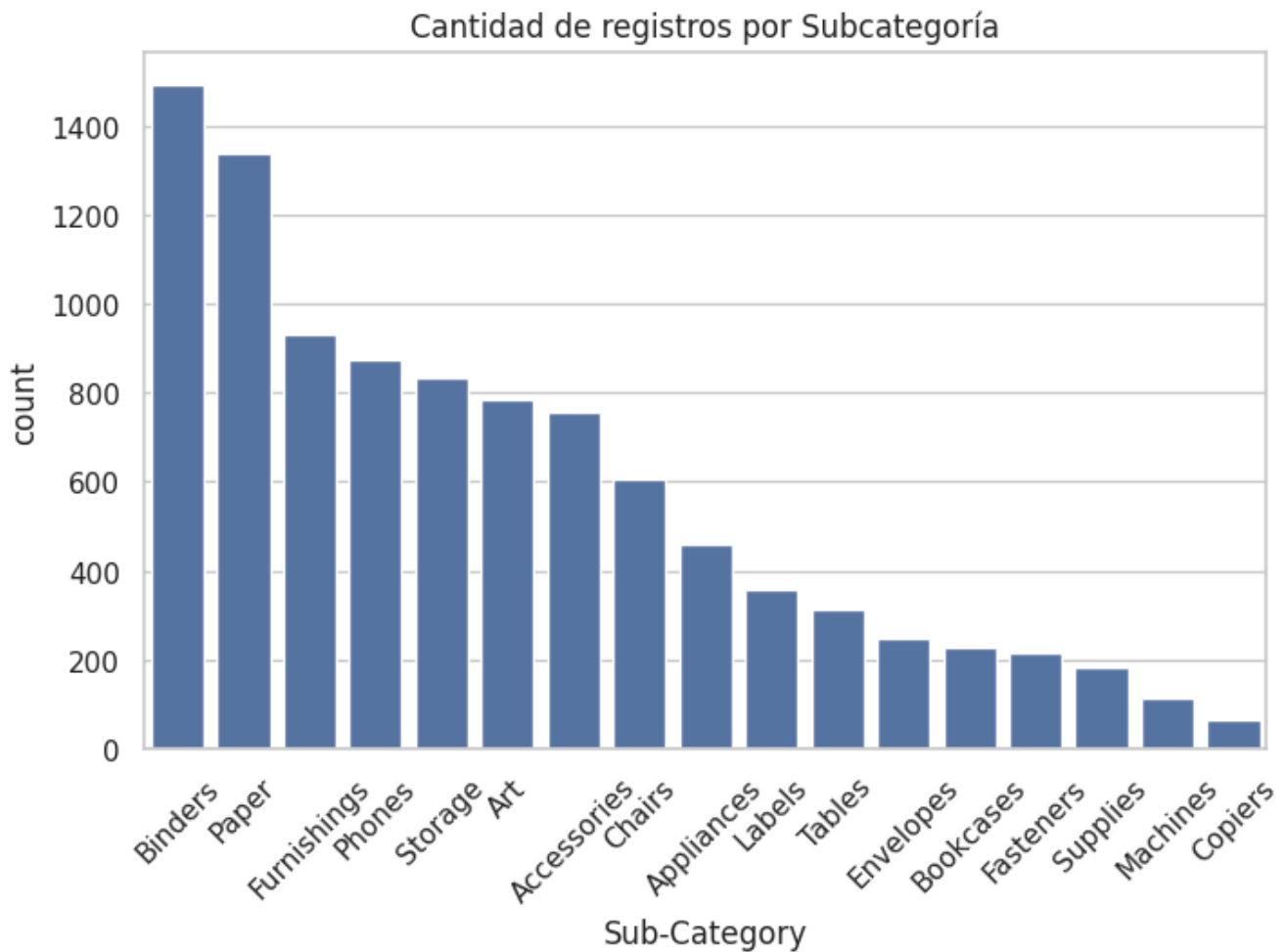
A veces no queremos sumar valores, solo queremos saber cuántas veces aparece cada elemento. Para eso usamos `countplot()`. Es una forma rapidísima de ver qué subcategorías tienen más movimiento.

Fijate que usamos `order` para que las barras aparezcan ordenadas de mayor a menor; esto hace que el gráfico sea mucho más fácil de leer que si estuvieran desordenadas.

```
# Contamos cuántos registros hay por cada subcategoría
sns.countplot(
    x='Sub-Category',
    data=df,
    order=df['Sub-Category'].value_counts().index)

# Rotamos las etiquetas del eje X 45 grados para que no se encimen los no.
plt.xticks(rotation=45)

plt.title('Cantidad de registros por Subcategoría')
plt.show()
```



Actividad práctica:

Análisis exploratorio del dataset Iris

En este ejemplo vamos a realizar un análisis exploratorio de datos sobre el dataset **Iris**, un conjunto clásico en ciencia de datos.

- **Fuente del Dataset:** [Iris \(Kaggle\)](#).

El objetivo será:

- Usar Pandas para cargar un dataset real.
- revisar la estructura general del dataset,
- analizar los tipos de datos de cada columna,
- detectar posibles valores nulos o duplicados,
- y construir algunas visualizaciones sencillas para comprender mejor la información.

```

import kagglehub
import os
import pandas as pd

# Descargar la versión más reciente del dataset
path = kagglehub.dataset_download("uciml/iris")

# Ver qué archivos hay dentro de la carpeta descargada
# print(os.listdir(path)) muestra TODOS los archivos de la carpeta
archivos_csv = [archivo for archivo in os.listdir(path) if archivo.endswith(

print("Archivos CSV encontrados:")
print(archivos_csv)

```

Using Colab cache for faster access to the 'iris' dataset.
Archivos CSV encontrados:
['Iris.csv']

Lectura del archivo

Ahora cargamos el archivo CSV en un DataFrame. A partir de este momento, ya podremos comenzar el análisis exploratorio.

```

# Cargar el primer archivo CSV encontrado
ruta_csv = os.path.join(path, archivos_csv[0])
df = pd.read_csv(ruta_csv)

# Mostrar las primeras filas
df.head()

```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

Primer vistazo al dataset

Antes de hacer análisis más específicos, conviene observar algunas filas del conjunto de datos para reconocer sus columnas y el tipo de información que contiene.

```
# Mostrar algunas filas al azar
df.sample(5)
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
40	41	5.0	3.5	1.3	0.3	Iris-setosa
92	93	5.8	2.6	4.0	1.2	Iris-versicolor
60	61	5.0	2.0	3.5	1.0	Iris-versicolor
33	34	5.5	4.2	1.4	0.2	Iris-setosa
27	28	5.2	3.5	1.5	0.2	Iris-setosa

Estructura general del dataset

En esta parte vamos a revisar la forma general del dataset: cuántas filas y columnas tiene, cuáles son sus nombres de columnas y qué tipo de datos aparece en cada una.

```
# Cantidad de filas y columnas
print("Dimensiones del dataset:", df.shape)
```

Dimensiones del dataset: (150, 6)

```
# Nombres de las columnas
print("Columnas del dataset:")
print(df.columns.tolist())
```

Columnas del dataset:
['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']

```
# Información general del DataFrame
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   Id              150 non-null   int64  
 1   SepalLengthCm   150 non-null   float64
 2   SepalWidthCm    150 non-null   float64
 3   PetalLengthCm   150 non-null   float64
 4   PetalWidthCm    150 non-null   float64
 5   Species         150 non-null   object  
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

A partir de esta salida podemos reconocer cuántos registros hay, qué columnas están presentes y qué tipos de datos utiliza pandas en cada caso.

Esto es importante porque, antes de analizar o modelar datos, necesitamos saber si las variables están correctamente interpretadas como numéricas, categóricas o de texto.

Valores nulos y registros duplicados

Un paso clave del análisis exploratorio consiste en verificar si faltan datos o si existen filas repetidas. Estas situaciones pueden afectar los resultados del análisis y, en algunos casos, exigir una limpieza previa.

```
# Conteo de valores nulos por columna
df.isnull().sum()
```

	0
Id	0
SepalLengthCm	0
SepalWidthCm	0
PetalLengthCm	0
PetalWidthCm	0
Species	0

dtype: int64

```
# Cantidad de filas duplicadas
print("Cantidad de filas duplicadas:", df.duplicated().sum())
```

Cantidad de filas duplicadas: 0

Si el dataset tuviera valores nulos o duplicados en cantidad significativa, sería necesario decidir cómo tratarlos.

En este caso, vamos a observar si el conjunto de datos se encuentra limpio o si presenta algún detalle a corregir antes de continuar.

Resumen estadístico de las variables numéricas

Ahora vamos a observar medidas descriptivas básicas, como media, desvío estándar, mínimo, máximo y cuartiles. Esto nos permite conocer mejor cómo se distribuyen los valores de las variables numéricas.

```
df.describe()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

Este resumen ayuda a tener una primera idea del comportamiento de las medidas del dataset.

Por ejemplo, podemos comparar los rangos de valores entre largo y ancho de sépalos y pétalos, y empezar a detectar si alguna variable presenta valores llamativos o muy dispersos.

Identificación de variables categóricas

Además de las variables numéricas, también es importante reconocer las columnas categóricas. En este dataset, la especie de cada flor funciona como una categoría y será útil más adelante para comparar grupos y colorear visualizaciones.

```
# Ver valores únicos de la variable categórica
print("Valores únicos en la columna species:")
print(df["Species"].unique())
```

```
Valores únicos en la columna species:
['Iris-setosa' 'Iris-versicolor' 'Iris-virginica']
```

```
# Cantidad de registros por especie
df["Species"].value_counts()
```

Species	count
Iris-setosa	50
Iris-versicolor	50
Iris-virginica	50

dtype: int64

Con esta información podemos ver si las clases están equilibradas o si alguna especie aparece con mucha mayor frecuencia que las demás.

Esto también resulta importante al momento de construir visualizaciones o pensar en análisis posteriores.

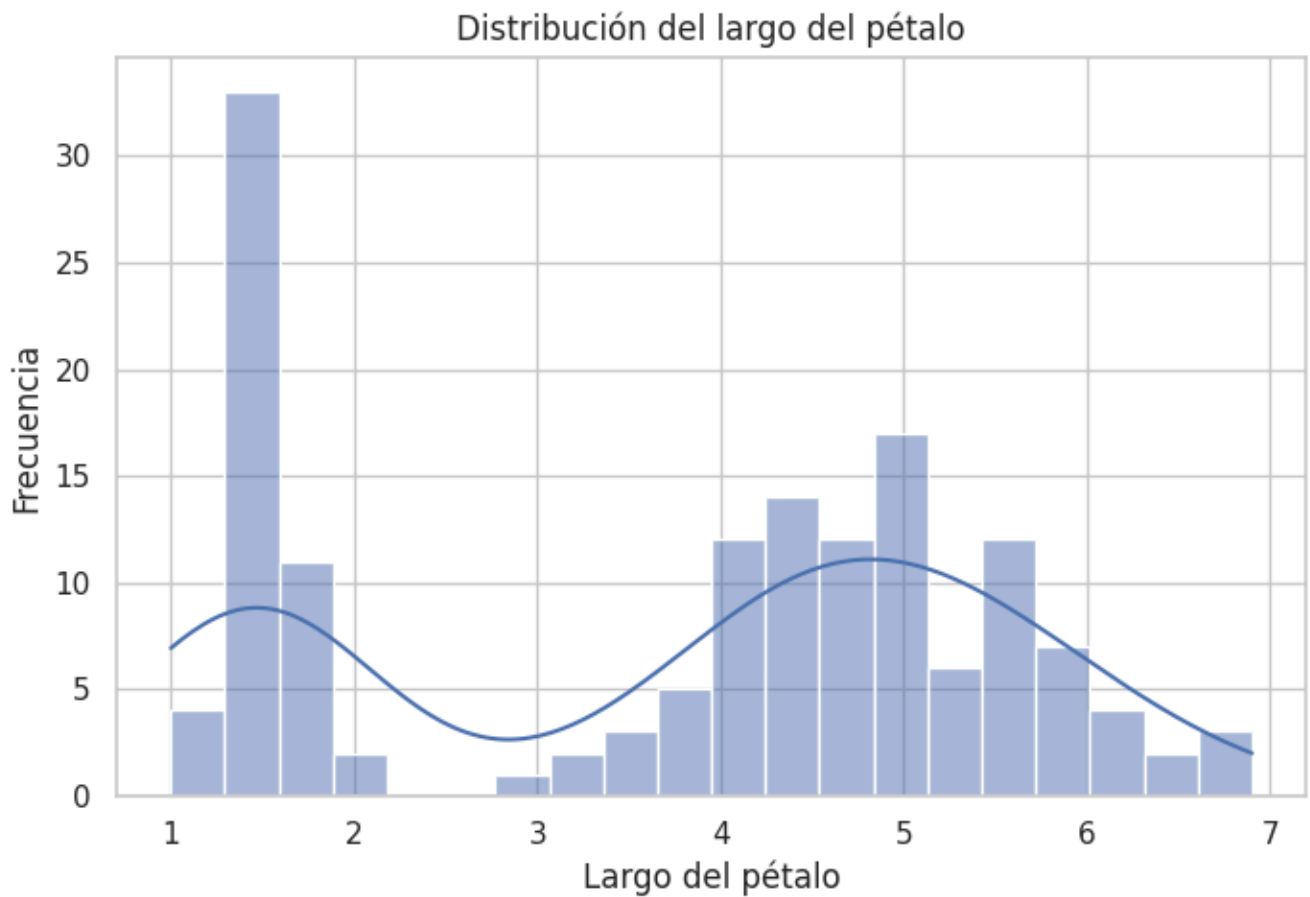
Visualización 1: distribución de una variable numérica

Una buena forma de comenzar a visualizar los datos es observar cómo se distribuye una variable numérica.

En este caso, vamos a trabajar con el largo del pétalo. Un histograma nos permitirá ver en qué rangos se concentran más observaciones.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(8, 5))
sns.histplot(data=df, x="PetalLengthCm", bins=20, kde=True)
plt.title("Distribución del largo del pétalo")
plt.xlabel("Largo del pétalo")
plt.ylabel("Frecuencia")
plt.show()
```

Este gráfico permite detectar la forma general de la distribución.

Además, aparecen varios picos. Eso puede sugerir que las distintas especies no se comportan igual respecto de esta variable.

Visualización 2: relación entre dos variables

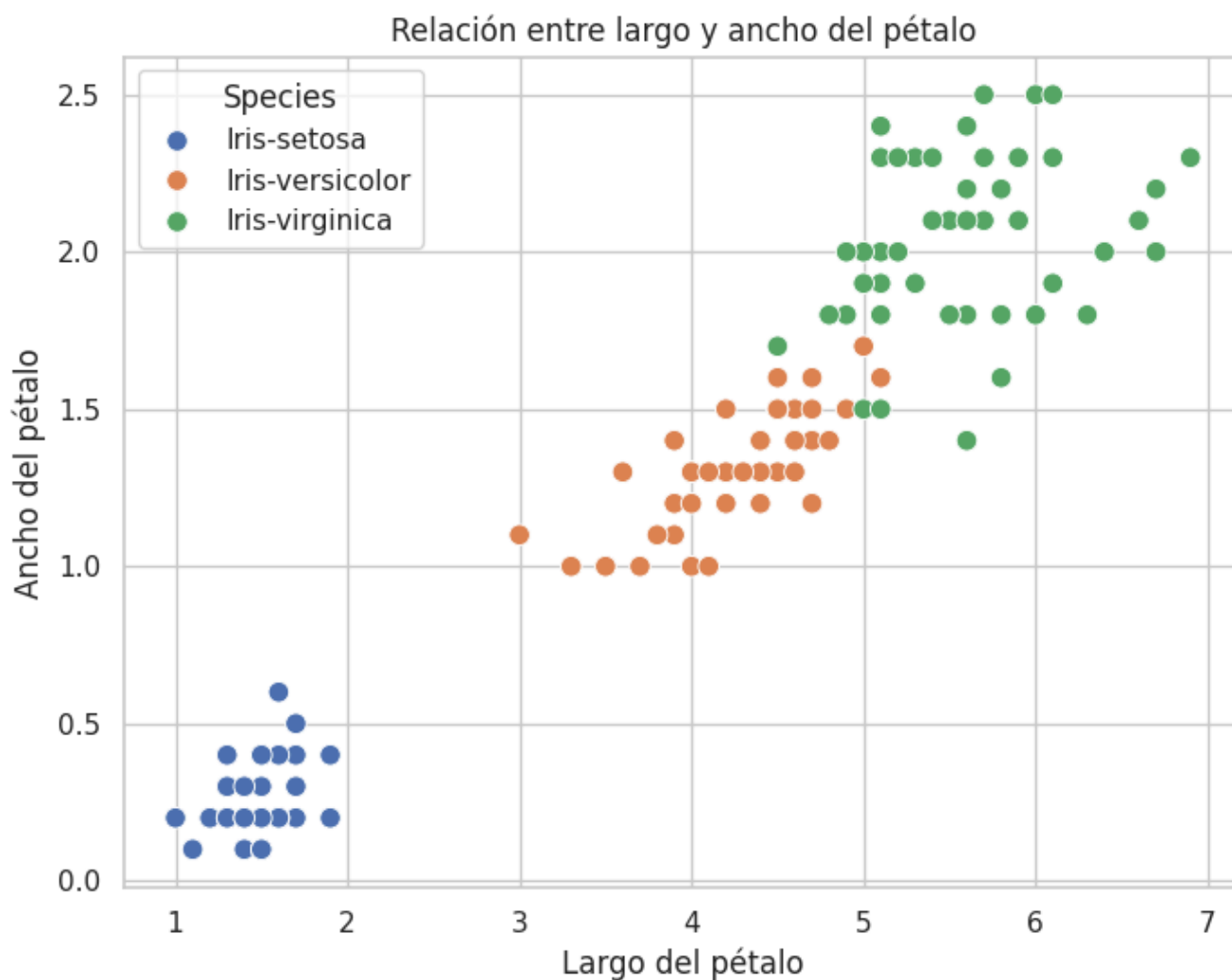
Ahora vamos a representar la relación entre dos variables numéricas mediante un gráfico de dispersión.

Usaremos el largo y el ancho del pétalo, y además diferenciaremos los puntos según la especie para ver si aparecen agrupamientos.

```
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=df,
    x="PetalLengthCm",
    y="PetalWidthCm",
    hue="Species",
    s=70
)

plt.title("Relación entre largo y ancho del pétalo")
plt.xlabel("Largo del pétalo")
plt.ylabel("Ancho del pétalo")
```

```
plt.show()
```



En este gráfico se puede observar si existe relación entre ambas variables y si las especies forman grupos distinguibles.

Cuando los puntos de distintas categorías se separan con cierta claridad, eso sugiere que esas variables pueden ser útiles para diferenciar clases.

Visualización complementaria: comparación por especie

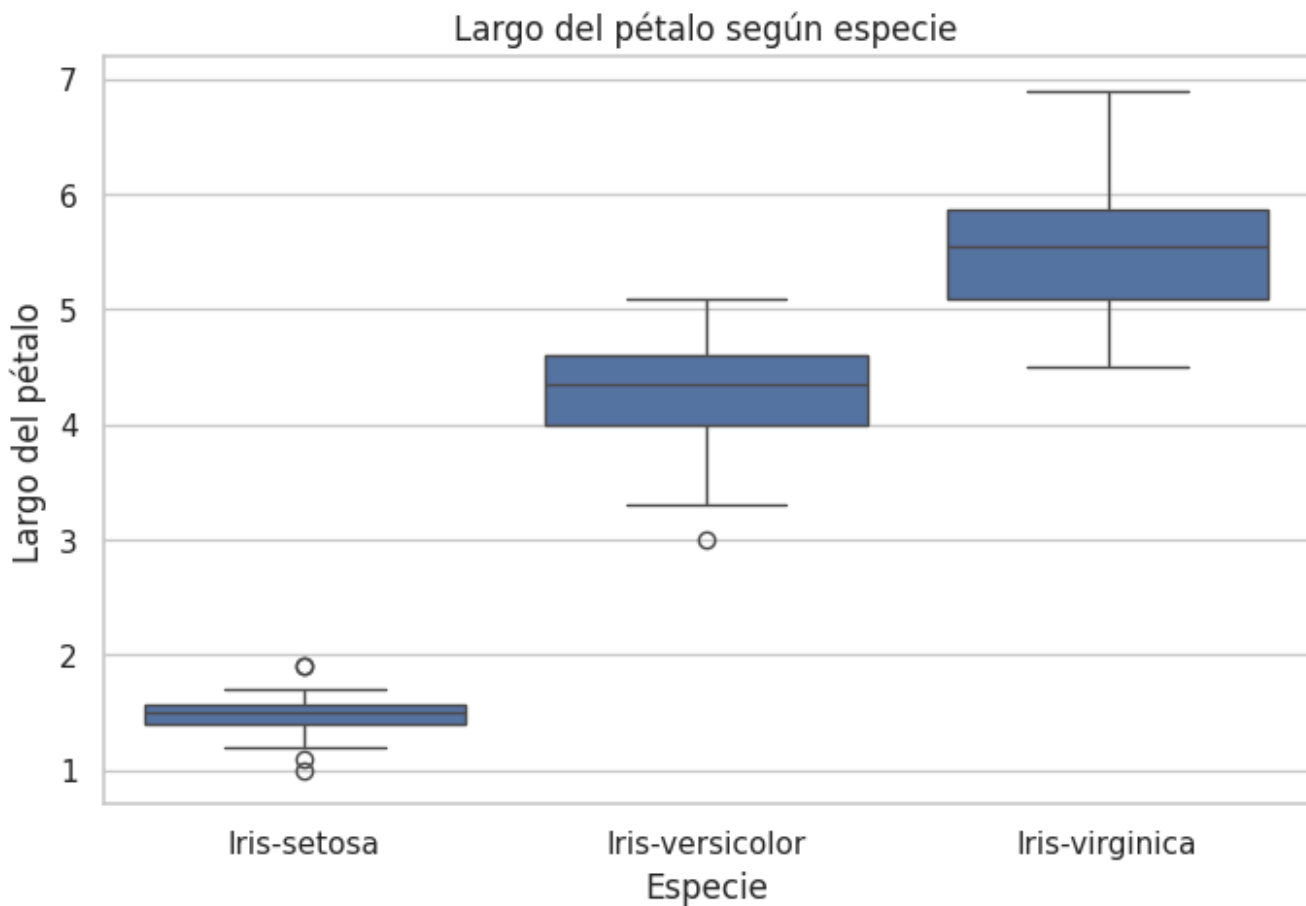
Además del gráfico de dispersión, también es útil comparar una variable numérica entre categorías.

Para eso podemos usar un boxplot, que resume la distribución de cada grupo y permite comparar medianas, dispersión y posibles valores extremos.

```
plt.figure(figsize=(8, 5))
sns.boxplot(data=df, x="Species", y="PetalLengthCm")

plt.title("Largo del pétalo según especie")
```

```
plt.xlabel("Especie")
plt.ylabel("Largo del pétalo")
plt.show()
```



Este tipo de gráfico ayuda a comparar rápidamente cómo varía una medida numérica entre grupos.

En particular, permite ver si una especie tiende a tener valores más altos o más bajos que las otras, y si presenta mayor o menor dispersión.

Registro de hallazgos

Después de revisar la estructura del dataset y construir algunas visualizaciones, conviene dejar por escrito una breve síntesis de lo observado.

Documentar hallazgos es una parte importante del análisis exploratorio, porque ayuda a interpretar los datos antes de avanzar hacia tareas más complejas.

Hallazgos preliminares

A partir del análisis realizado, podemos observar que el dataset Iris presenta una estructura simple y ordenada, con una cantidad acotada de registros y un conjunto reducido de variables.

Las columnas numéricas corresponden a medidas de sépalos y pétalos, mientras que la columna `Species` identifica la especie de cada flor y funciona como variable categórica.

No se detectaron valores nulos en las columnas analizadas, y el dataset se presenta, en principio, bastante limpio para trabajar. Esto facilita el análisis y reduce la necesidad de tareas de limpieza previas.

En las visualizaciones se observa que algunas variables, especialmente las relacionadas con los pétalos, parecen diferenciar bastante bien a las especies. En particular, el gráfico de dispersión entre largo y ancho del pétalo muestra agrupamientos que sugieren una separación clara entre categorías.

Cierre final

El análisis exploratorio de datos constituye una etapa fundamental dentro de cualquier trabajo de ciencia de datos. Antes de intentar extraer conclusiones o construir modelos, es necesario conocer la estructura del dataset, revisar su calidad y comenzar a interpretar el significado de sus variables.

En la segunda parte de este cuaderno, el dataset Iris nos permitió poner en práctica ese proceso de manera clara y accesible. A partir de la exploración inicial y de las visualizaciones construidas, pudimos reconocer patrones, comparar categorías y describir de manera general el comportamiento de los datos.

Este tipo de trabajo no solo ayuda a comprender mejor un conjunto de datos particular, sino que también permite desarrollar criterios y hábitos de análisis que serán útiles en proyectos más complejos.